Lab-1

Submission:

- Upload your submission (per group of two) as ***one zip file*** on **Canvas by 11 pm on Wednesday 1 April**.
  (Late penalties: up to 24h late: -20% of achieved marks, more than 24h late: 0 marks).
- **The zip file should include:**
  - A short report (one to two pages) to explain about the design space explorations' part with a table to indicate the performance (as the execution time or as no. of clock cycles) for some cases with different instruction and data cache sizes and explaining about the highest performance configuration.
    Name and ID of both team members should be written on top of the report.
  - All the required software (C files)
  - All the design files for the hardware generation of part 3 (or part 4) so the system can be synthesized.

Introduction

The purpose of this lab is to learn:

1) **A.** How to design a system based on Nios II processor using Altera (Intel) FPGA **Platform Designer** in Quartus and implement it on DE1-SoC FPGA board (or DE2-115).
   B. How to run C/C++ applications on the developed system.
2) How to use high resolution Interval Timer to measure the performance of a Nios II based system for a given application (or part of the application).
3) How to extend the previously designed system with additional I/O Ports.
4) How to explore the possibilities to improve the performance of the system. For example, finding the optimum cache sizes (for both instruction and data caches) for the given application.

Part 1A – System Hardware Design *(1 Mark)*

Open Quartus and create a new project. You can use any name for the project. For the following discussion we consider the project name as ***`Nios_Sys_1`***.

- For DE1-SoC: Choose DE1-SoC board to use Cyclone V FPGA as the target device (or choose Cyclone V device 5CSEMA5F1C6).
- For DE2-115: Choose Cyclone IV E, Device EP4CE115F29C7.

To design a Nios II processor system open Platform Designer in Quartus from Tools menu `Tools > Platform Designer`. Then you will have the following screen (IP Catalog is highlighted):

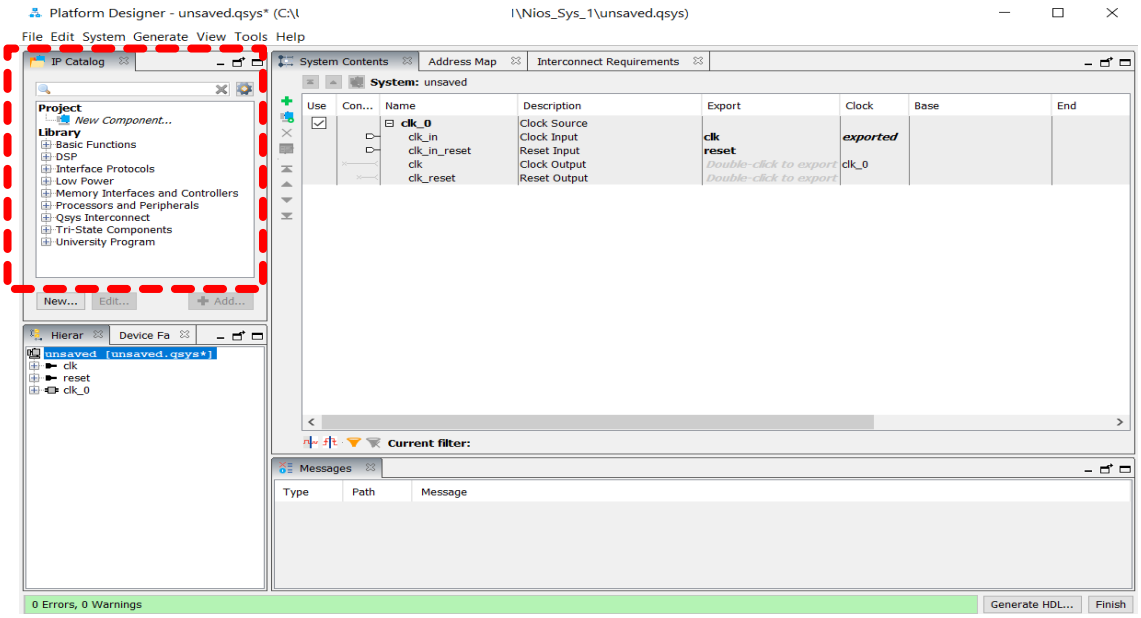


Figure 1: Platform Designer opening screen

For the clock source, we'll change clk_0 to clk as shown in Figure 2.

From the IP Catalog, we choose Processors and Peripherals and then Embedded Processors.

We are going to instantiate a Nios II core (**Nios II/f**) with the following parameters from the **IP Catalog**:

- 4KB Instruction Cache, 2KB Data Cache
- For Arithmetic Instructions: `Auto Selection` for Multiply/Shift/Rotate

- In `Advanced Features` choose *0x00000001* for CPUID control register value
- Use the default values for the other settings. We'll specify the required parameters (vector addresses) in `Vectors` later.

We can use any name for the system components. In this document, we use name ***cpu*** for our Nios II core as shown in Figure 2.

All system components are chosen from the **IP Catalog.** Next, we instantiate the following components with the required parameters:

- On chip RAM: 20 KB (20480 B), single port (use default setting for the other parameters)
- Interval Timer (using the default values: Timeout period: 1 ms, Timer counter size 32 bits)
- JTAG UART (using the default values)

After connecting the components properly, the next step is to assign the memory address for ***Reset*** Vector and ***Exception*** Vector after specifying which memory is used for Nios II core instructions. (In our case it is *onchip_memory* as shown in Figure 2). We use the default values here. However, it should be assigned based on the system memory map.

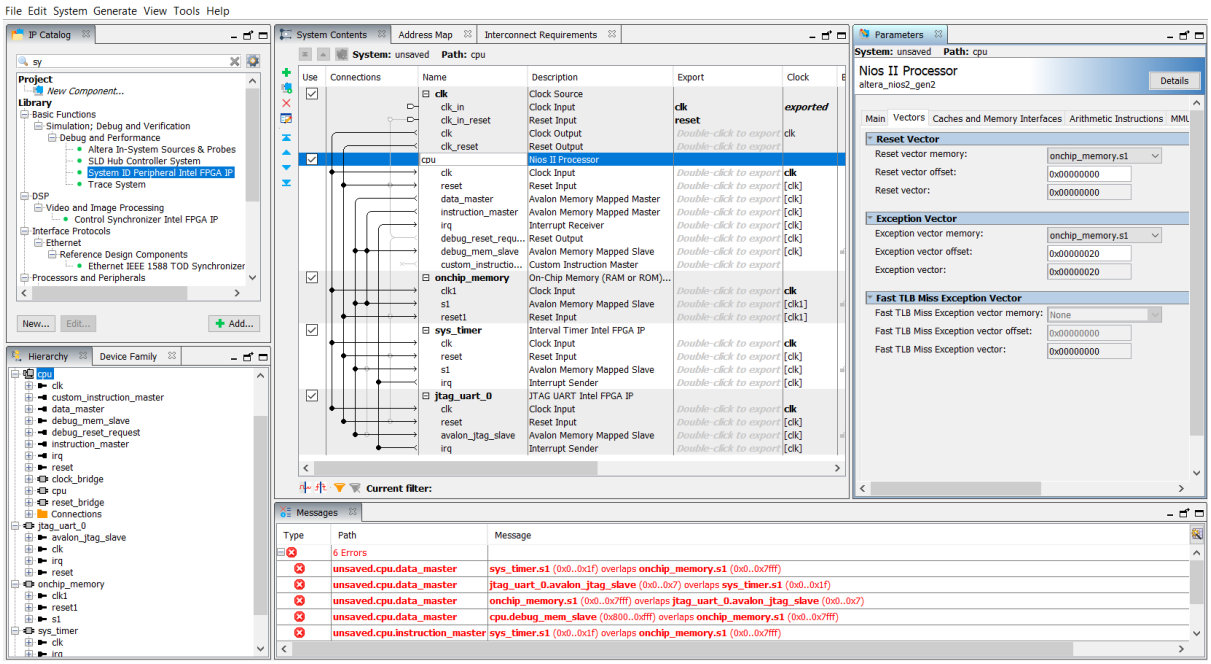


Figure 2: Instantiated components and their interconnection (incomplete)

Now we have a system like Figure 3 (with our specified name for the components). Don't worry about the error messages now and we'll deal with that later.

Note that both the Timer and JTAG UART may generate interrupt on CPU ***irq*** inputs. So, we should assign proper **irq** to each one based on their priority for interrupt service. (We consider the timer with a higher priority and assign **irq0** to it. JTAG UART will be assigned for example, **irq5**.)

As shown in Figure 3, the assigned memory addresses conflict and that's why we have error messages. We can assign the base address for each component based on our *pre-planned system memory map* or allowing *Platform Designer to assign the base memory addresses automatically* as shown in Figure 4. We won't have error messages after assigning the memory addresses (Figure 5).

Then, we save our designed system (as `Nios_V1.qsys` in our example).

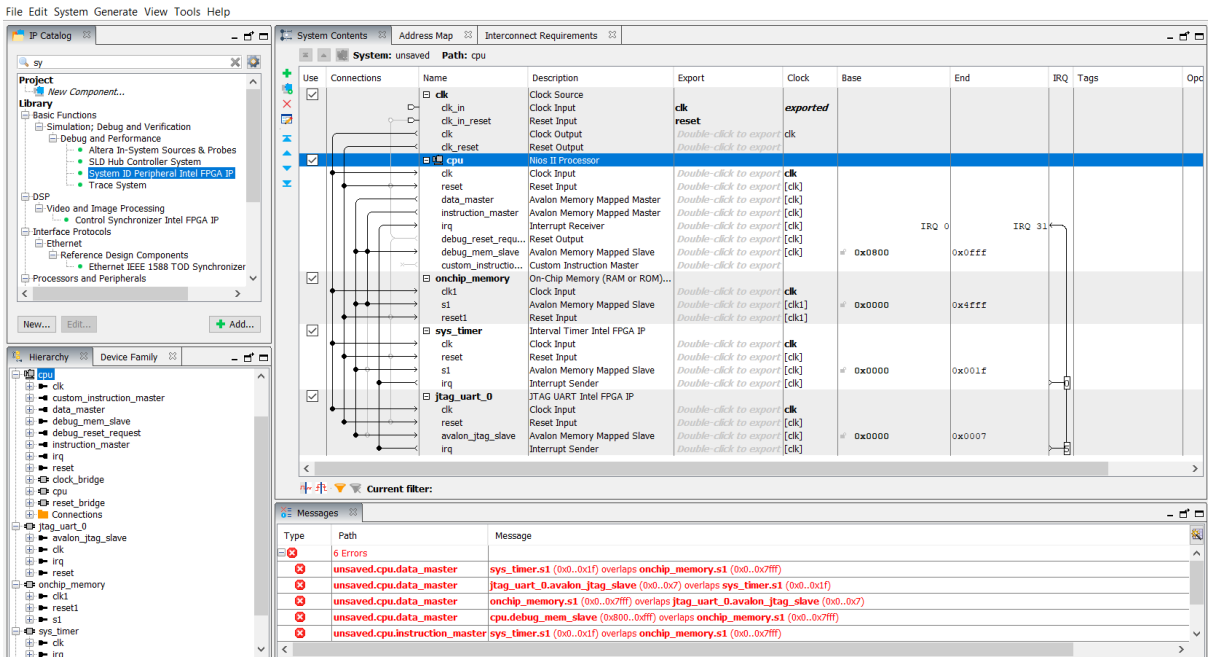


Figure 3: Instantiated components and their interconnection (incomplete)

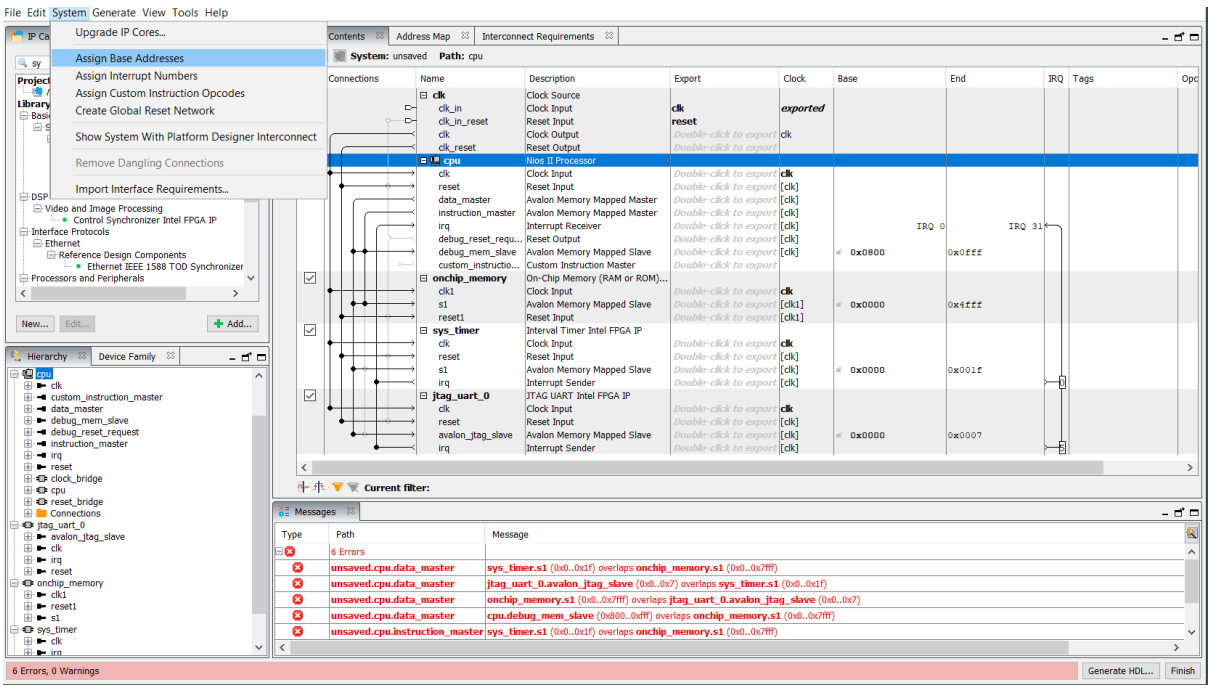


Figure 4: Automatic base memory address assignment

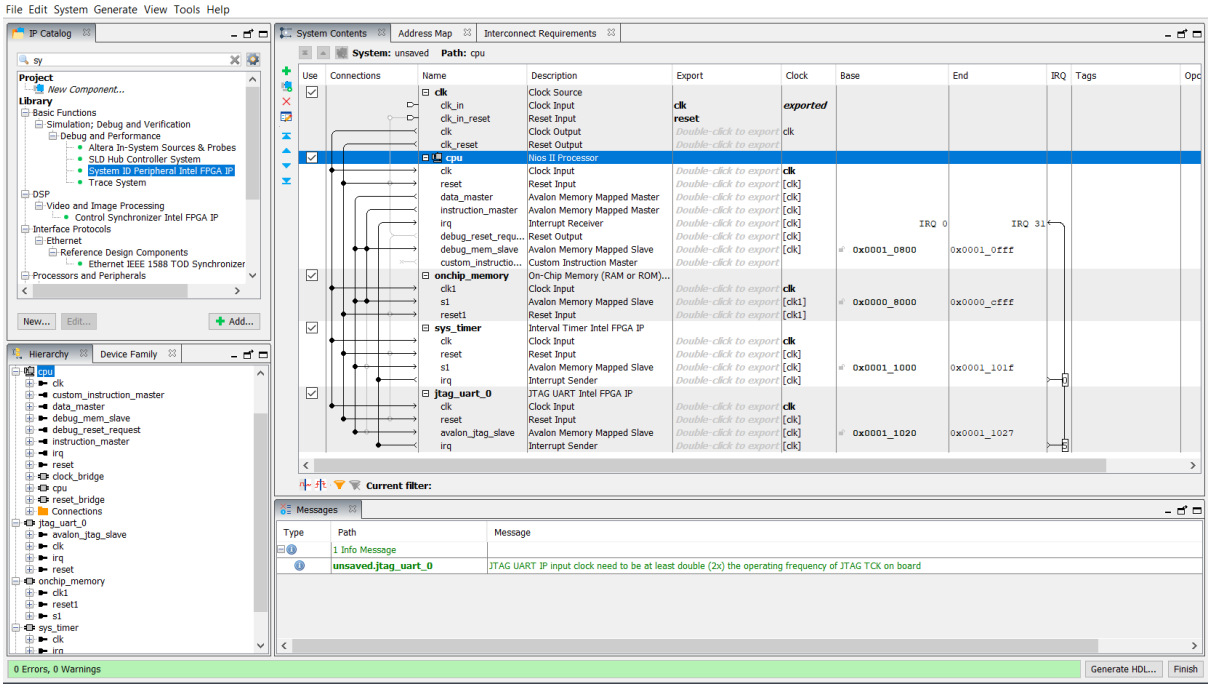


Figure 5: Instantiated components and their interconnection (with assigned memory addresses)

When the system design is completed, it can be saved (in our case as Nios_V1.qsys).

To display binary values on LEDs, we need to add PIO component, which is configured as 8-bit output port as shown in Figure 6. (Note: if you closed Platform Designer after saving the system in previous step, now open file Nios_V1.qsys to add the new component to the system).

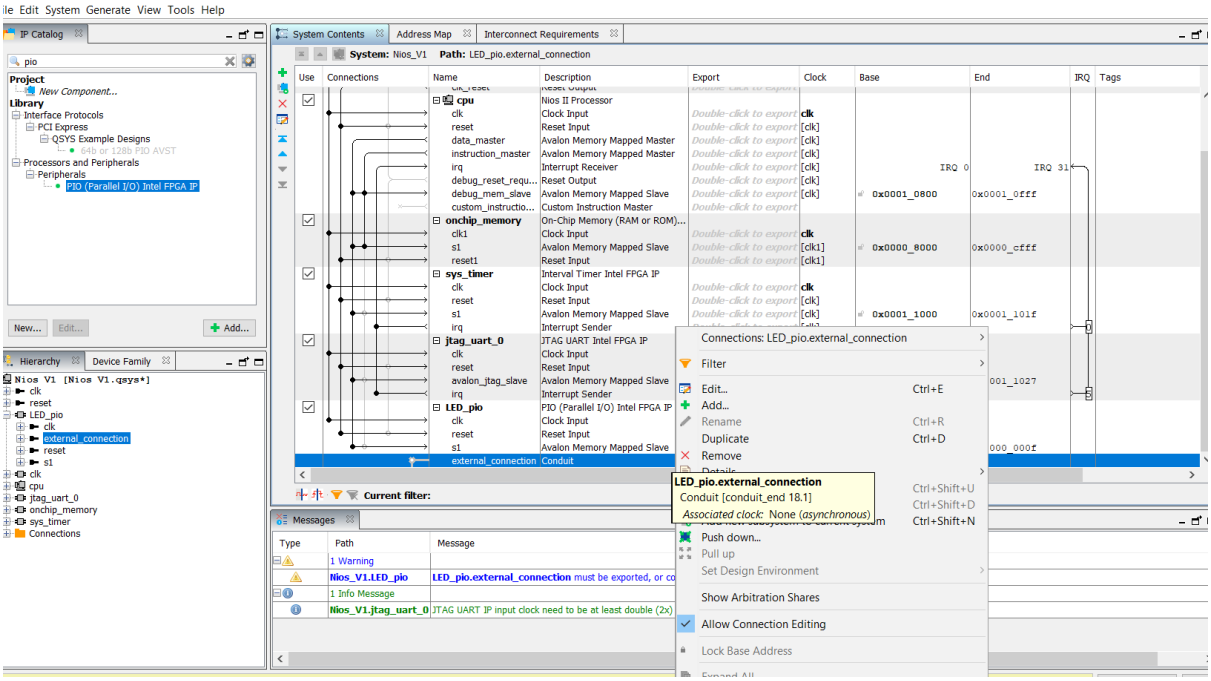


Figure 6: Adding PIO for LEDs connection

We should make LED_pio an external port (as shown in Figure 7). Then, if needed we should assign the base address for the new component (or using automatic assignment shown in Figure 8).

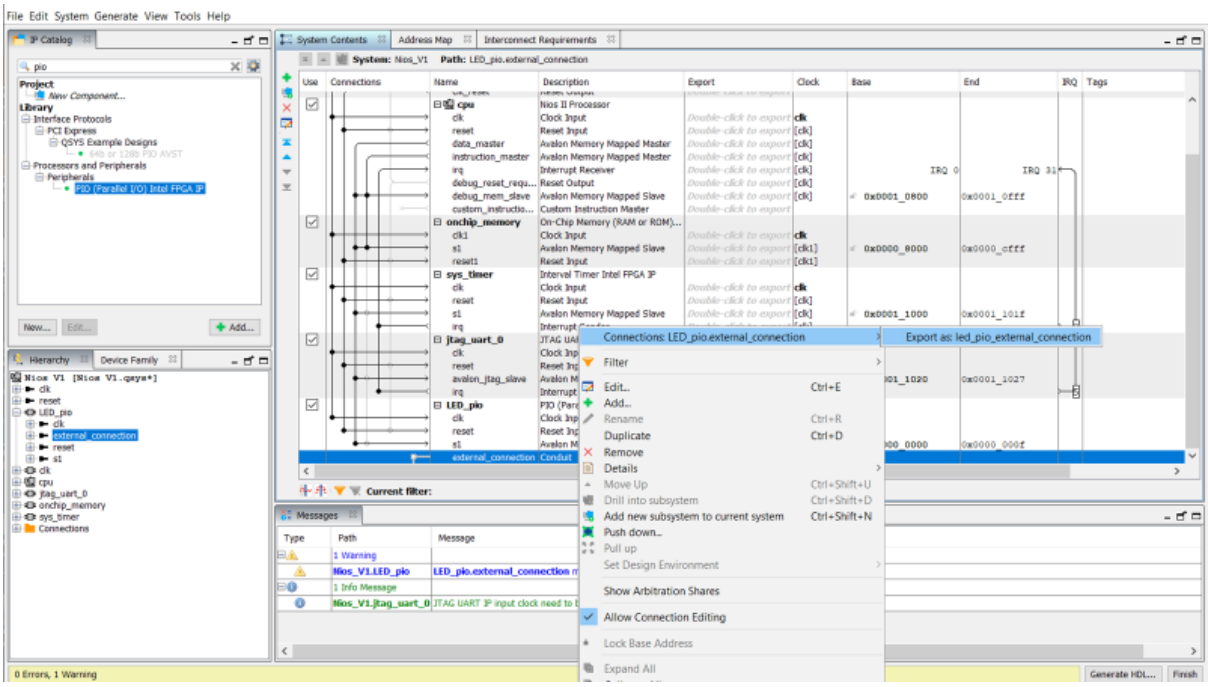


Figure 7: Adding PIO for LEDs connection (exported as output port)

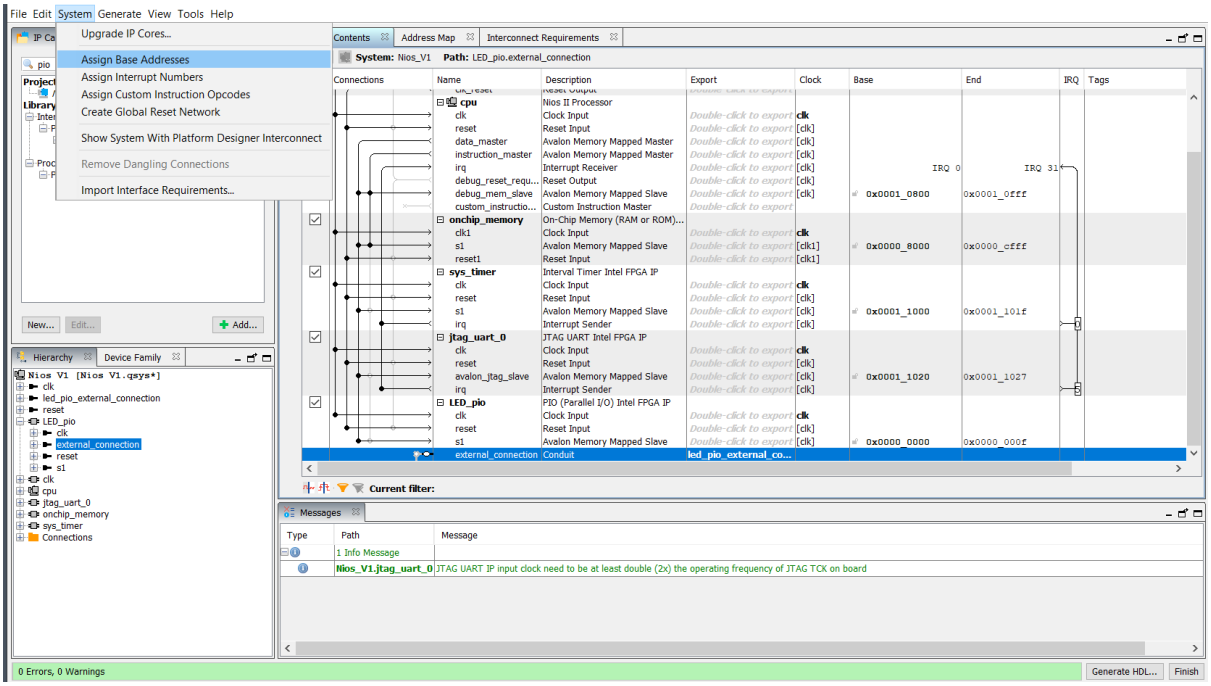


Figure 8: Re-assigning the base memory addresses (if needed)

At this stage, our system is similar to Figure 9.

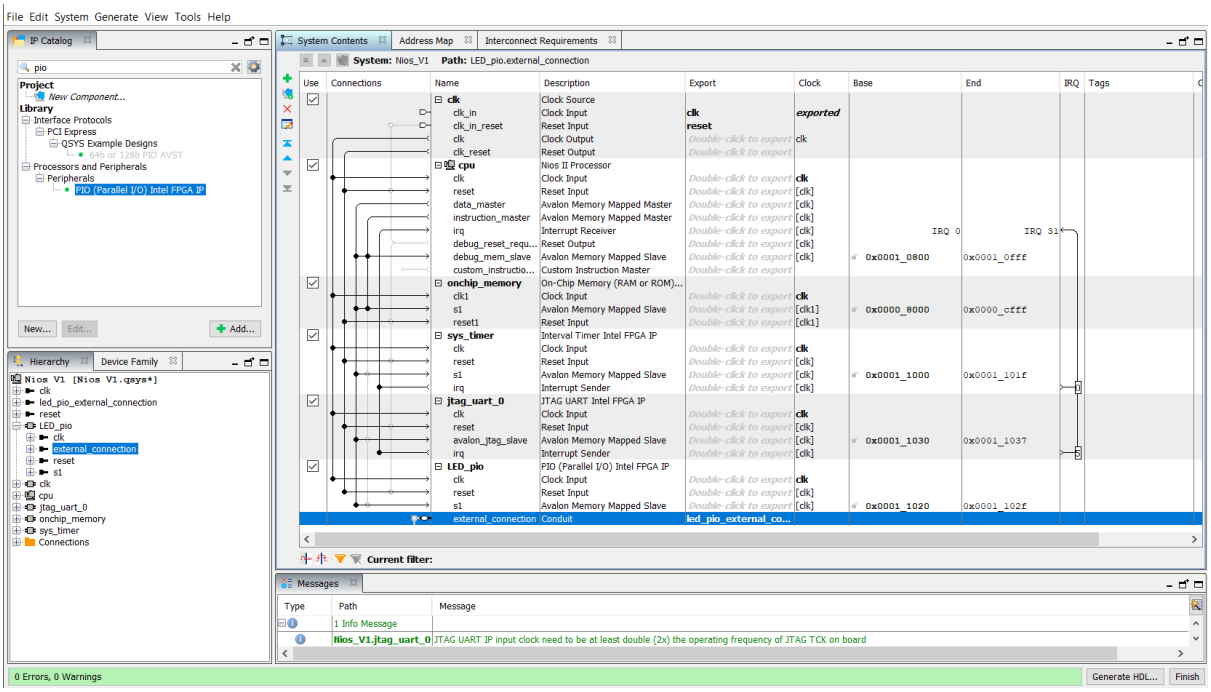


Figure 9: Completed Nios II based system in Platform Designer

We'll save the designed system (in Nios_V1.qsys). The final step is to generate the HDL files. In our case, we choose VHDL as shown in Figure 10 and `Generate`. After that we can close Platform Designer.

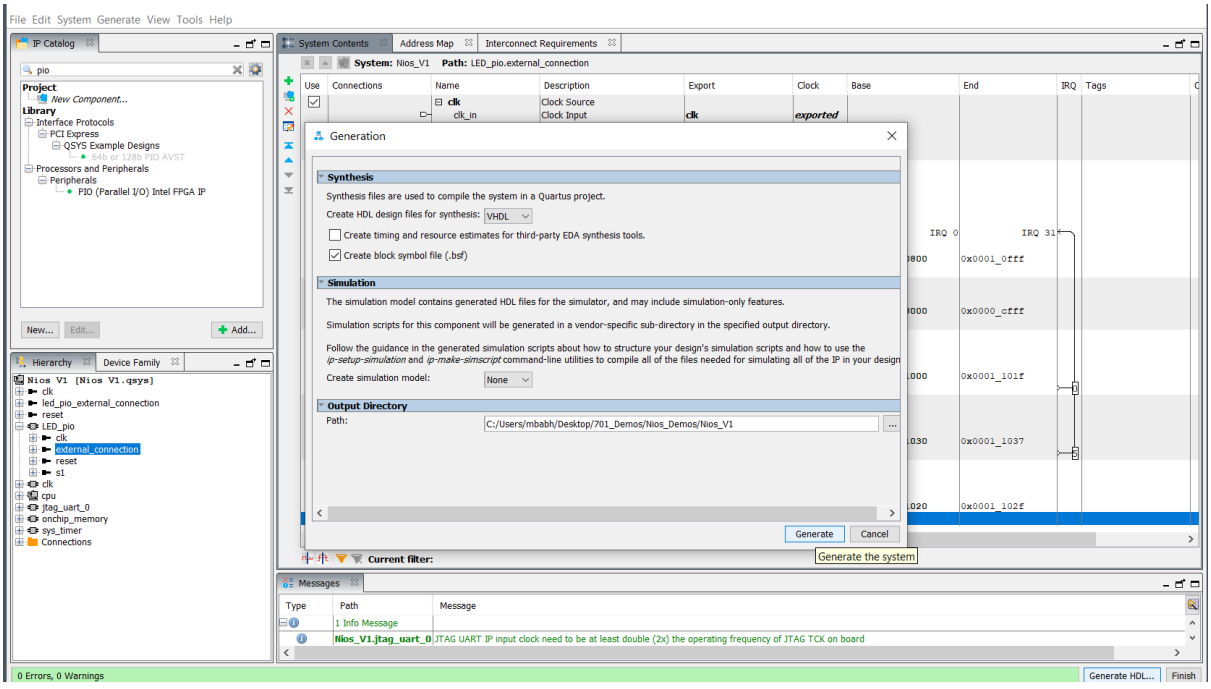


Figure 10: Generating HDL file (with top level in VHDL) for the Nios II system in Platform Designer

For the top level, we can either use the schematic diagram and instantiate our Nios II based system symbol and make the required connection. The other option is to write the top level VHDL code and instantiate the Nios II system as a component.

Platform Designer generated *Nios_V1_inst.vhd* which can be used as a template for component instantiation in top level entity VHDL code in Quartus project. We'll write the top level VHDL code (Figure 11) and add it to the project. Note that we use the proper name for input and output ports based on the required DE1-SoC (or DE2-115) pin assignment.

Also, *Nios_V.qip* (from synthesis folder) should be added to the project for our Nios II system `Assignments > Settings > Files > Add ...`

```vhdl
-- Top level VHDL File

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity Nios_Sys_1 is
    port (CLOCK_50 : in std_logic;
          KEY : in std_logic_vector(0 to 0);  -- Note here for reset
          LEDR : out std_logic_vector(7 downto 0));
end entity Nios_Sys_1;

architecture structure of Nios_Sys_1 is
    component Nios_V1 is
        port (
            clk_clk                           : in  std_logic                     := 'X'; -- clk
            reset_reset_n                     : in  std_logic                     := 'X'; -- reset_n
            led_pio_external_connection_export : out std_logic_vector(7 downto 0)         -- export
        );
    end component Nios_V1;


begin
    u0 : component Nios_V1
        port map (
            clk_clk                           => CLOCK_50,
            reset_reset_n                     => KEY(0),
            led_pio_external_connection_export => LEDR
        );
end architecture structure;
```

Figure 11: An example of top level VHDL file

Use the given *DE1_SoC.qsf* file for DE1-SoC (or *DE2_115_pin_assignments.csv* for DE2-115) for pin assignment: `Assignments > Import Assignments` and choose *DE1_SoC.qsf* (or *DE2_115_pin_assignments.csv* for DE2-115). The final step of hardware design is to compile the system in Quartus: `Processing > Start Compilation` and then program the FPGA: `Tools > Programmer`. Note that proper .sof file should be selected.

The FPGA board should be connected to your computer with USB Blaster.

Part 1B – Running the Software *(1 Mark)*

We write C code to implement an 8-bit binary counter displayed on 8 Red LEDs on DE1-SoC (or DE2-115). (In our example, to make the changes on LEDs visible, we use a delay loop which iterates 3000000 times before displaying each count value on LEDs).

To develop the software in C/C++, we use Nios II Software Build Tools (SBT) for Eclipse. It can be opened in Quartus using `Tools > Nios II Software Build Tools (SBT) for Eclipse` as shown in Figure 12.

We specify the *workspace* … and then Eclipse windows open (Figure 13).

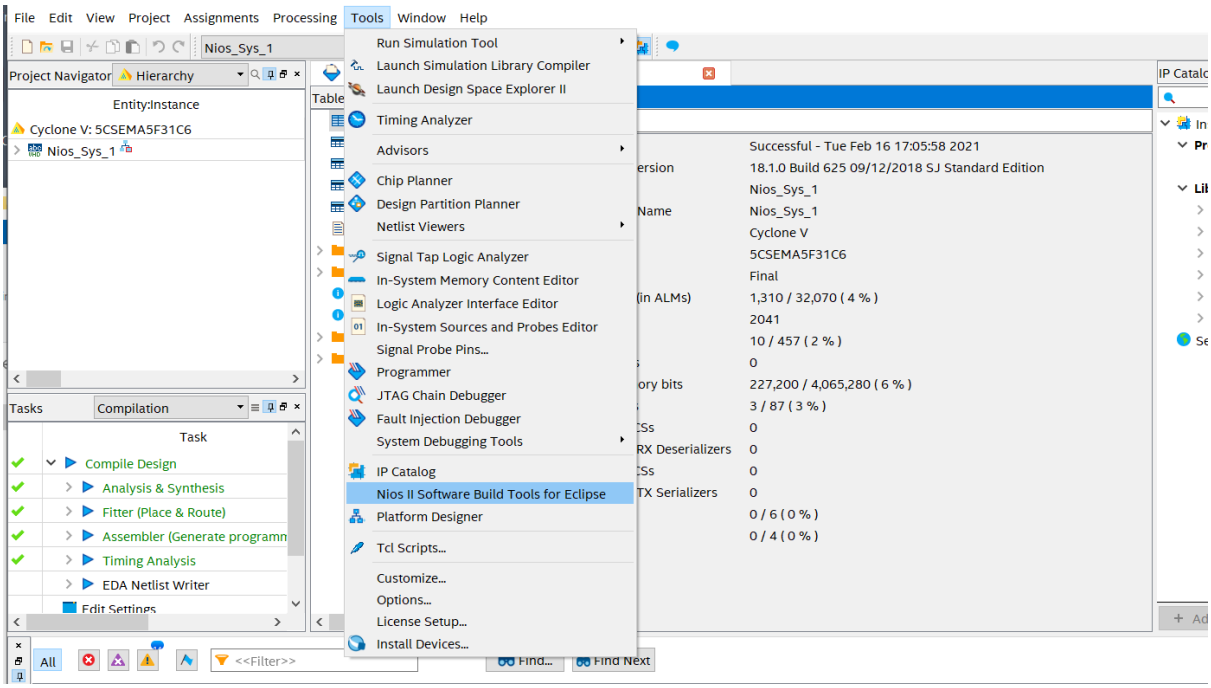


Figure 12: Open Nios II Software Build Tools (SBT) for Eclipse

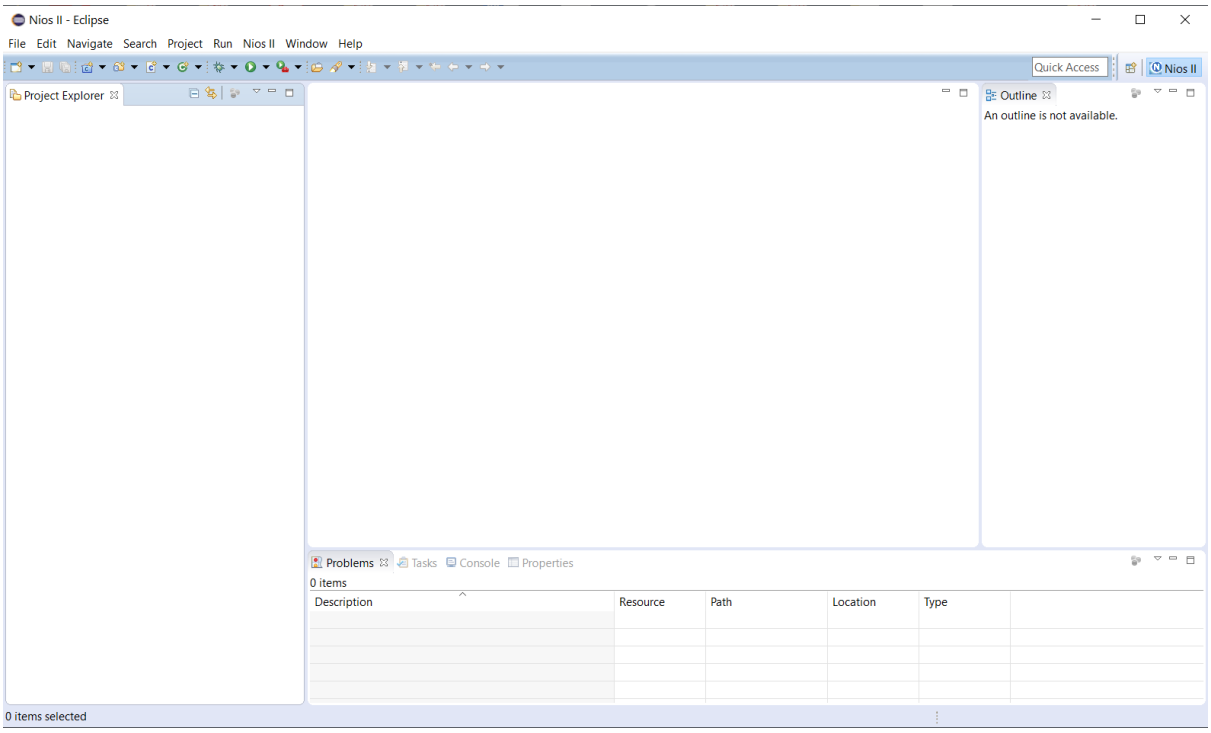


Figure 13: Nios II Software Build Tools (SBT) for Eclipse

We create a new application: `File > New > Nios II Application and BSP Template` and specify our hardware *`.sopcinfo`* file (Figure 14).

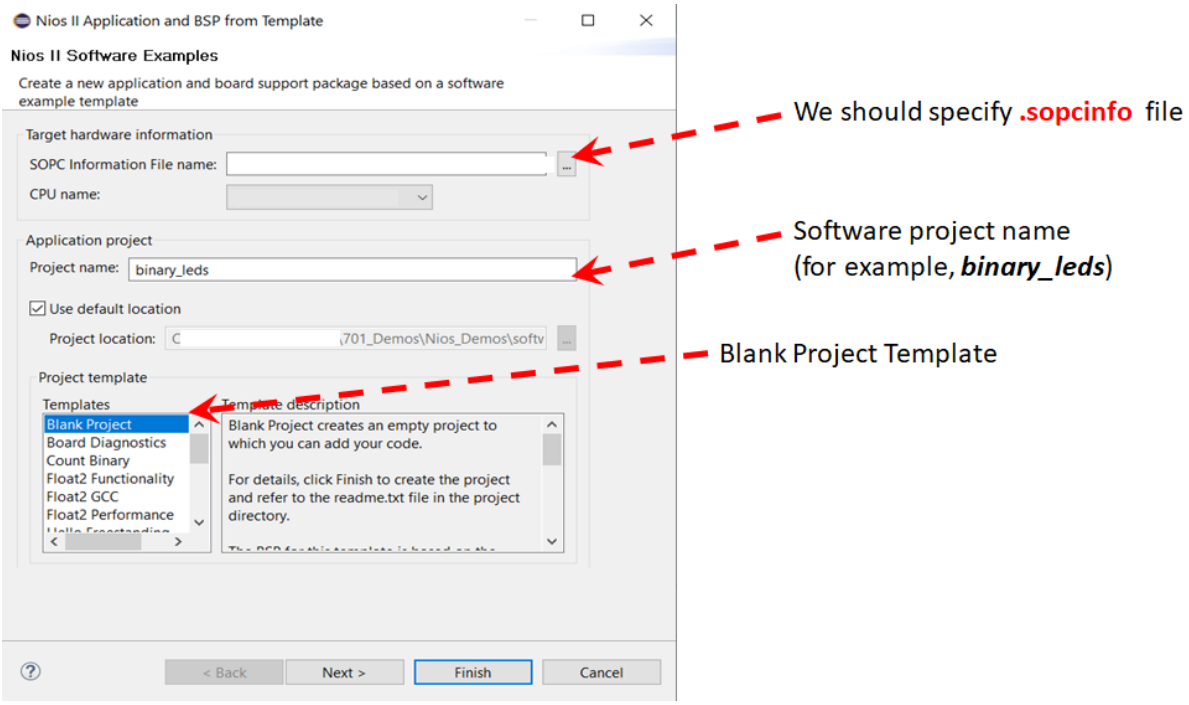


Figure 14: Setup the required Nios II Application and BSP information

The next step is to set the **BSP properties** (for example, to use ***small memory and reduced device drivers*** for our small application) as shown if Figures 15 and 16. (The other way is to use BSP Editor).

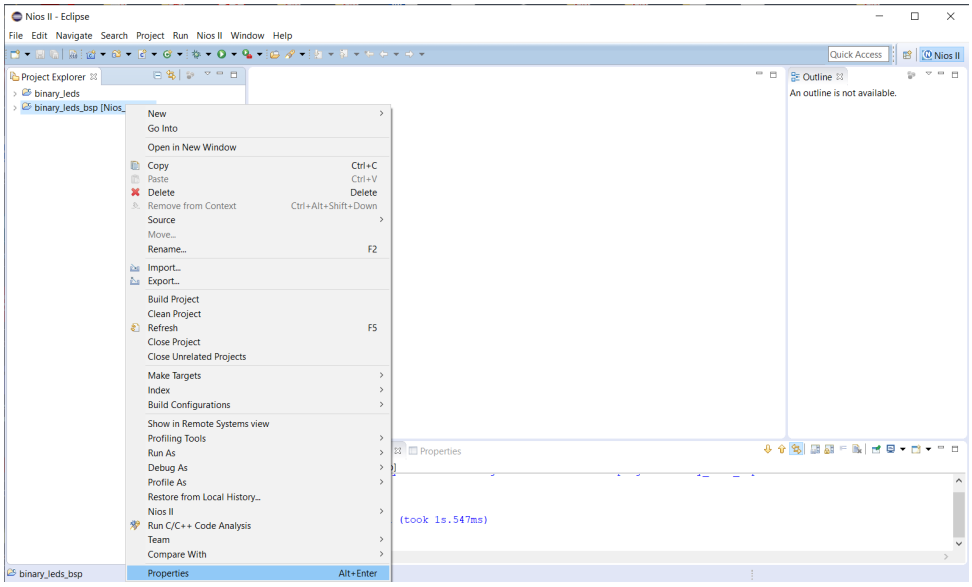


Figure 15: Setup BSP properties (menu)

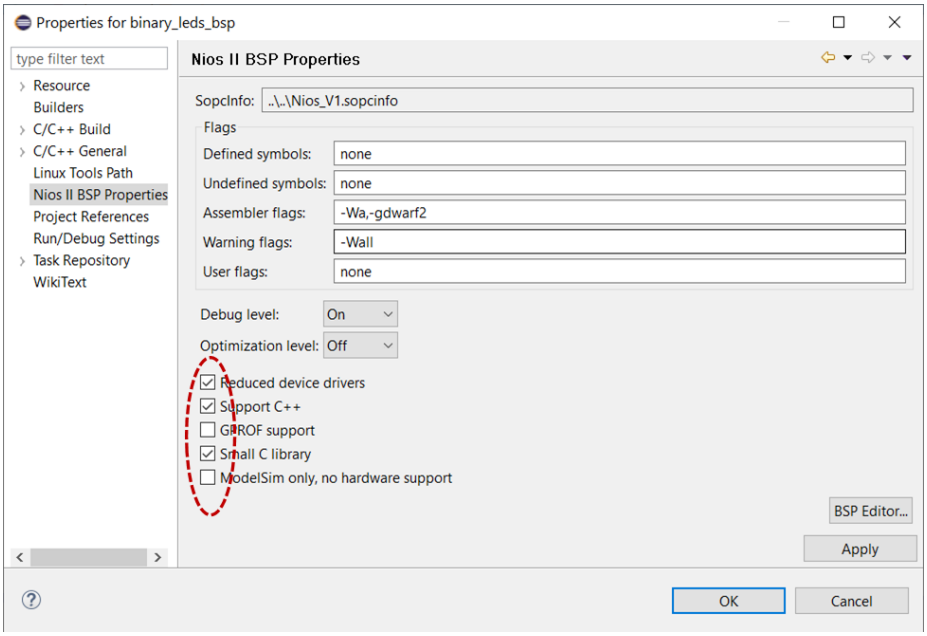


Figure 16: Setup BSP properties as required

We'll use C/C++ macros in our code to access the memory mapped components and peripherals (which are connected through Avalon-MM interfaces). To determine the proper macros, we can check files such as `system.h`, `altera_avalon_pio_regs.h` (for PIO) etc. as shown in Figures 17 to 19.

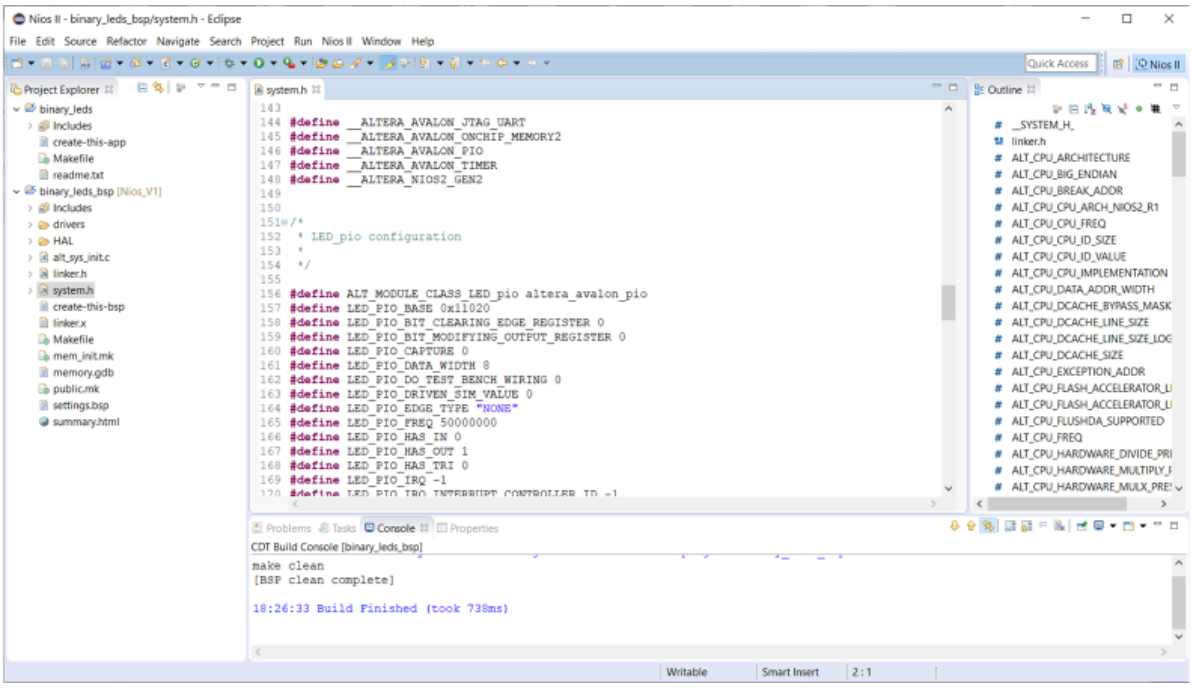


Figure 17: system.h (in BSP)

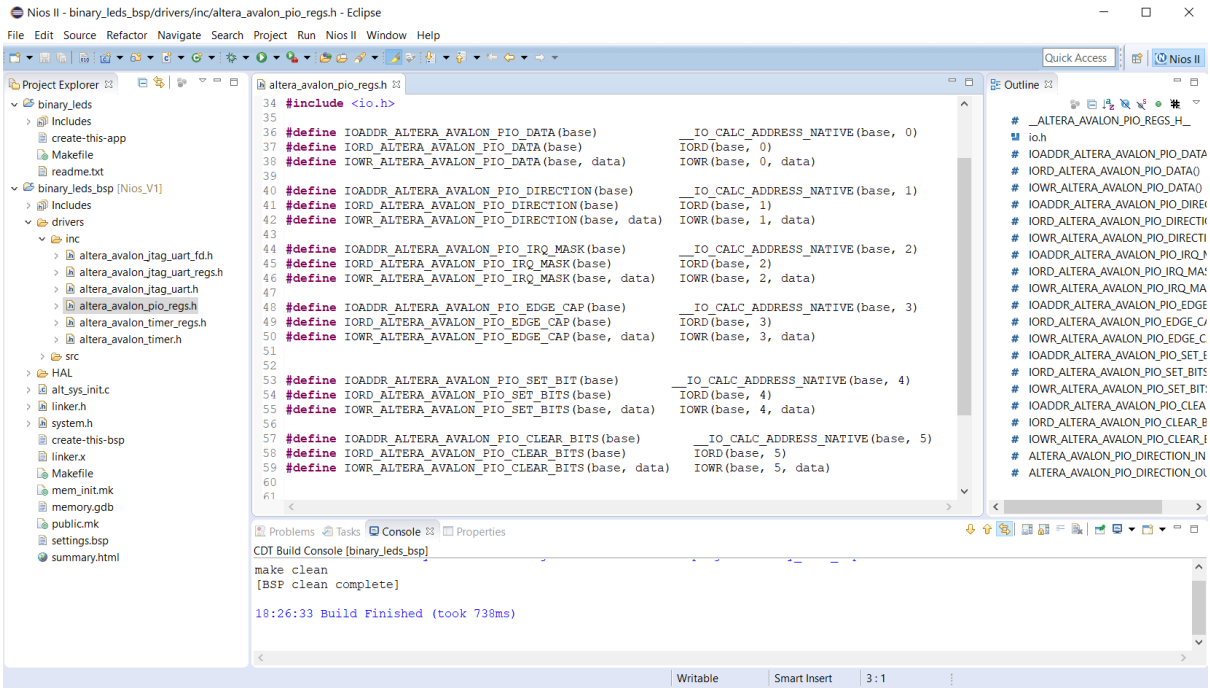
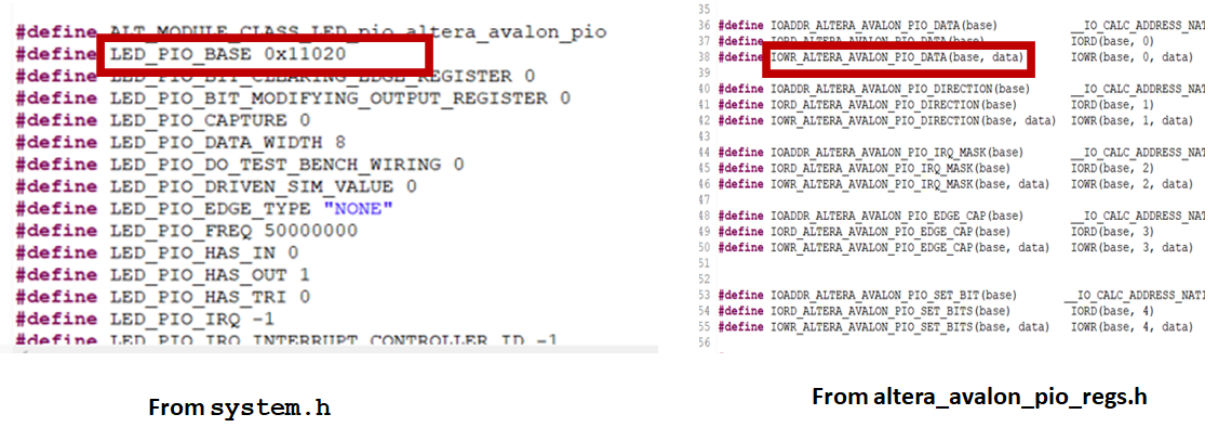


Figure 18: altera_avalon_pio.h in BSP drivers include files folder



From system.h

From altera_avalon_pio_regs.h

Figure 19: Identified macros for our application

We'll copy our **.c** file (for example, *my_binary_leds.c*) in application software folder (or develop it in Eclipse, which will be placed in software folder). The code for this example is in Figure 20.

```c
// Example: Compsys 701  (March 2022)
#include <stdio.h>
#include "system.h"
#include "altera_avalon_pio_regs.h"

int main()
{
    int i, count = 0;
    int Delay_Value = 3000000;

    printf("This is a simple binary counter displayed on LEDs.\n");

    while(1) {
        IOWR_ALTERA_AVALON_PIO_DATA(LED_PIO_BASE, count);
        // Delay for sometime
        for (i = 0; i < Delay_Value; i++) {
            ;
        }
        count++;
    }
    return 0;
}
```

Figure 20: Example C code.

The next step is to **Build Project** (Figure 21) and run the application (Figure 22).

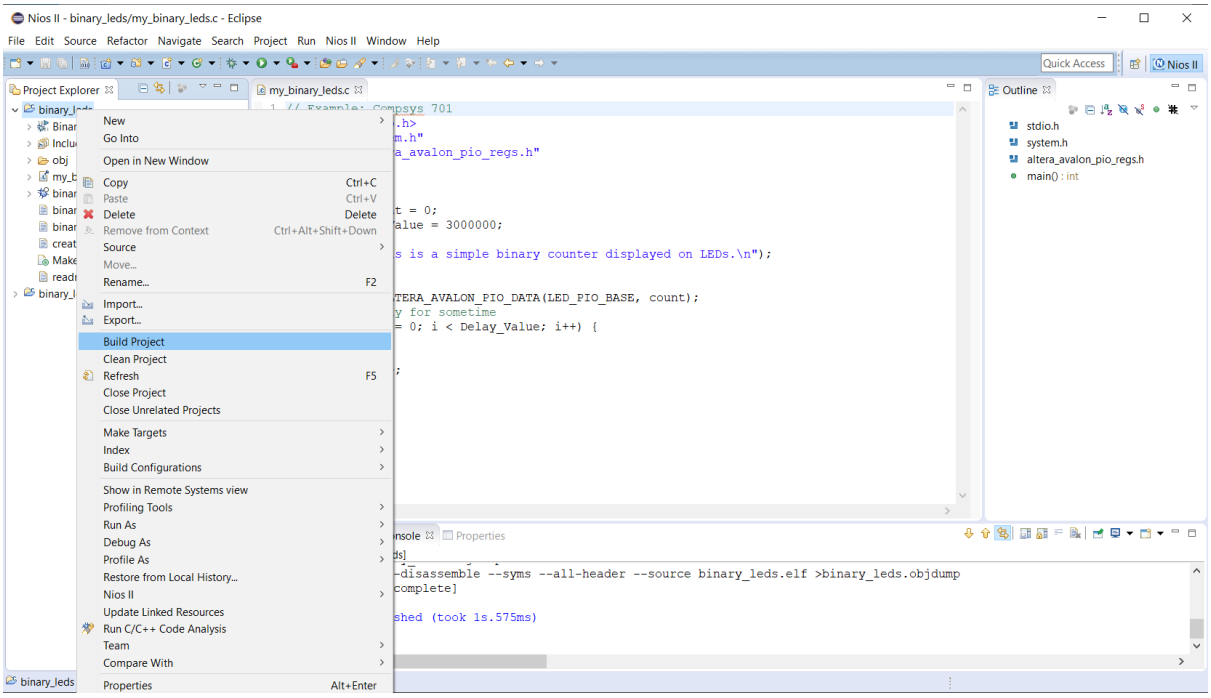


Figure 21: Build Project in Nios II Software Build Tools (SBT) for Eclipse

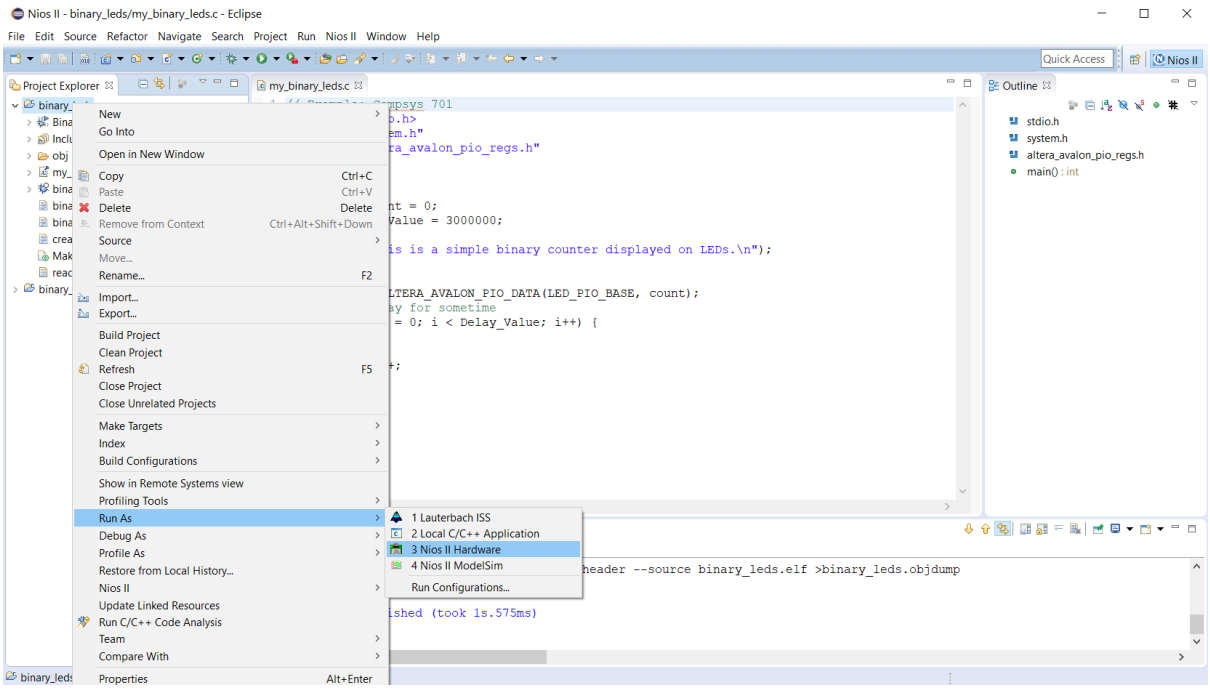
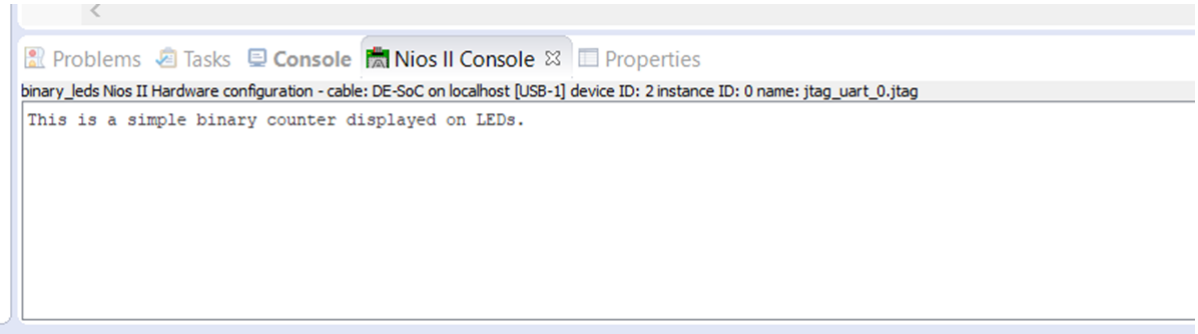


Figure 22: Run the application in Nios II Software Build Tools (SBT) for Eclipse

The text output (for *printf( )*) will be displayed on Nios II Console and the current count value on LEDs .



Part 2 – Performance Measurement *(2 Marks)*

In this part, we use high-resolution Interval Timer for performance measurement.

Implementing the hardware:

- Open Quartus and create a new project. You can use any name for the project (for example, **`Nios_Sys_2A`** or **`Nios_Sys_2B`** depending on using DE1-SoC or DE2-115 respectively. Choose the FPGA device as mentioned in Part 1A depending on the FPGA board that you use.
- Copy the given Qsys file (*`Nios_System_2A.qsys`* for DE1-SoC or *`Nios_System_2B.qsys`* for DE2-115) to your project folder. (In this system SDRAM is used to store the program code and data. However, SDRAM may be configured differently in DE1-SoC or DE2-115. You should check all the required components are included, otherwise should be added to the system).
- Open the given system (.qsys) in Platform Designer. (Make sure that there is no errors).
- Generate the HDL files. In our case, we choose VHDL and then **`Generate`**. After that keep the Platform Designer open as you need to make changes to the Nios processor configurations later.
- The next step is to instantiate the Nios II System in Quartus project. So, .qip (from synthesis folder) should be added to the project for our Nios II system, using *`Assignments > Settings > Files > Add ...`*
- For the top level, we can either use the schematic diagram and instantiate our Nios II based system symbol and make the required pin connections. The other option is to write the top level VHDL code and instantiate the Nios II system as a component. Use the given VHDL code *`Nios_Sys_2A.vhd`* (for DE1-SoC) or *`Nios_Sys_2B.vhd`* (for DE2-115) as a template (and make any required changes to the name of Nios II processor component) and then add it to the project. Note that we use the proper names for input and output ports based on the required FPGA board pin assignment.
- Use the given *`DE1_SoC.qsf`* file for DE1-SoC (or *`DE2_115_pin_assignments.csv`* for DE2-115) for pin assignment: *`Assignments > Import Assignments`* and choose *the proper pin assignment file*.
- The final step of hardware design is to compile the system in Quartus: *`Processing > Start Compilation`*. Then program the FPGA: *`Tools > Programmer`*. Note that proper **.sof** file should be selected.
  Note that the FPGA board should be connected to your computer with USB Blaster.

Running the software:

To develop the software in C/C++, we use Nios II Software Build Tools (SBT) for Eclipse. It can be opened in Quartus using *`Tools > Nios II Software Build Tools (SBT) for Eclipse`*.

We specify the *workspace* … and then Eclipse windows open.

We create a new application: `File > New > Nios II Application and BSP Template` and specify our hardware ***.sopcinfo*** file (as shown in Figure 23).

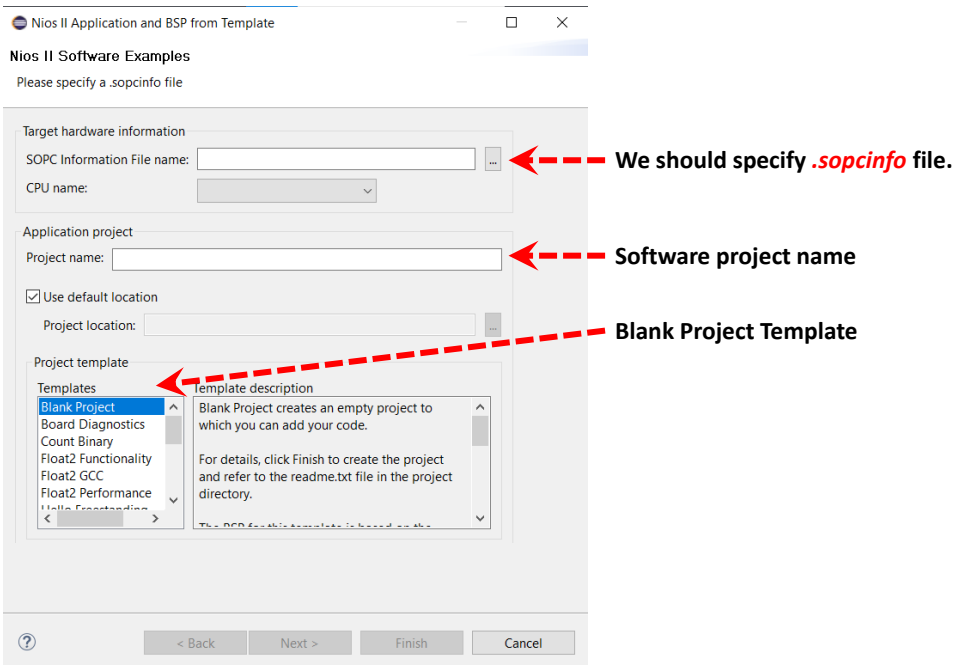


Figure 23: Setup the required Nios II Application and BSP information

The next step is to set the **BSP properties** to identify the **high-resolution timer** in the system required for performance measurements so the proper drivers can be generated automatically. For this purpose, we use **BSP Editor** and set the parameters (set `hal.timestamp_timer` to **sys_timer** and `hal.sys_clk_timer` to **none** as shown in Figures 24 and 25, and then use **Generate** to create the revised BSP and **Exit**.

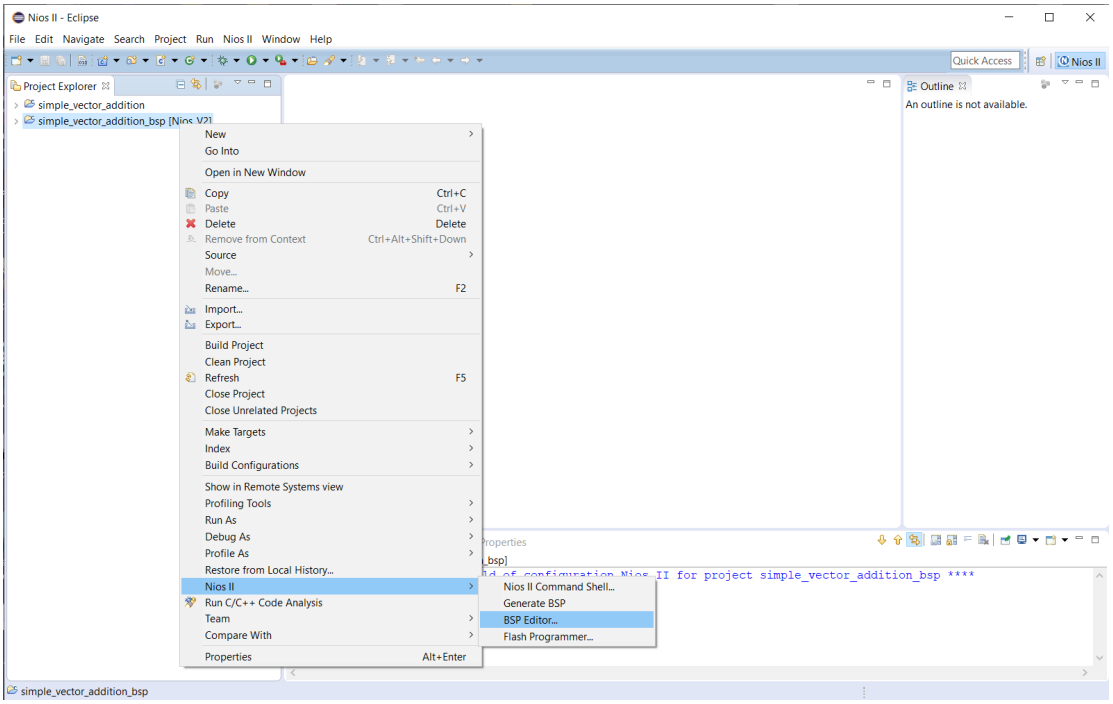


Figure 24: Editing BSP properties (menu)

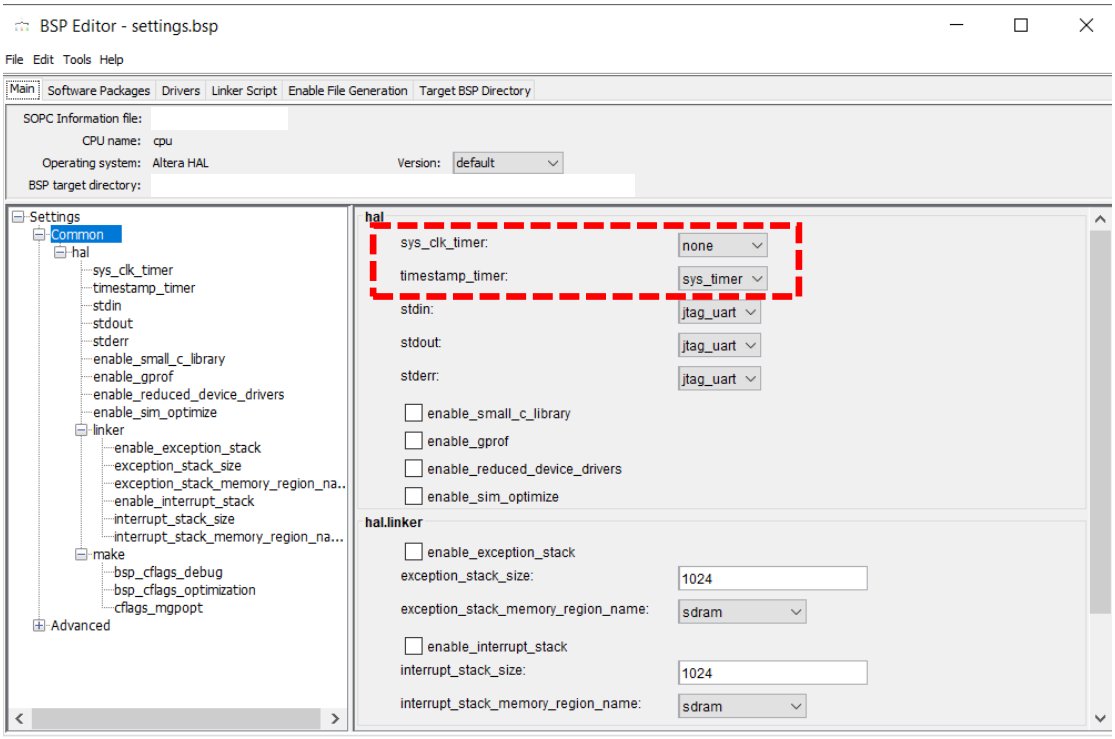


Figure 25: Setup BSP properties as required

Note that, as our application is not too big to fit in SDRAM memory (with the required libraries) therefore, we do not need to set *enable_small_c_library* and *enable_reduced_device_drivers* (as was done in Part 1A).

You can copy your **.c** file (for example, *my_binary_leds-V2.c*) in application software folder (or develop it in Eclipse, which will be placed in software folder. You may use Refresh if it cannot be seen in the application folder on Eclipse). The next step is to **Build Project** and run (**Run as Nios II Hardware**).

Part 3 – Extending the System with I/O Ports *(2 Marks)*

In this part, the given Nios II system (Nios_System_A1.qsys for DE1-SoC) is extended to add new PIOs.

a) These PIOs are: an 8-bit PIO for 8 switches on the FPGA Development board (SW0 to SW7), 2-bit PIO for two Buttons (KEY3 and KEY2), and PIOs for four seven-segment displays (each display for a single digit requires a 7-bit output port, or a combined larger output port can be used for all digits).

Note that the bits order for seven segment display is shown as follows (bit 0 is the least significant bit). We use high-resolution Interval Timer for performance measurement.

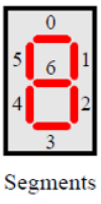


Figure 26: Seven Segment Display

b) Write the C code to count binary numbers from 00 to 99 (and again start from 00 after reaching 99). The output should be displayed on both Red LEDs (as binary values) and in Decimal format using the two right-most seven-segment displays. The two left-most seven-segment displays should show the last two digits of your upi as a Decimal number (for example if upi is abcd123 then 23 is displayed).

Part 4 – Improving System Performance *(4 Marks)*

The purpose of this part is to measure the performance of an application (or part of the application) and improve the performance by finding an optimum (or near optimum) size for instruction and/or data caches.

a) Write the C code for function Matrix_Operations, which performs addition, subtraction, or multiplication of two NxN matrices (A and B), with 32-bit integer elements and write the result in matrix C. The operation to be performed is determined by the values read through two switches (SW1 to SW0, where SW0 represents the least significant bit). The values read from these switches should be shown on the two left-most seven-segment displays.

SW1	SW0	Operation
0	0	Minimum Value
0	1	Addition
1	0	Subtraction
1	1	Multiplication

At the start of your application, the following two messages should be printed on the Console.

"This application performs NxN matrix operations."

"N is xxx, the operation is yyy."

xxx should be the current value of **N**, and yyy should be a text (either: "Minimum Value", or "Addition", or "Subtraction", or "Multiplication" depending on the current values of SW1 and SW0). If N < 2, then a proper error message should be printed on the Console and the program terminates without performing any operations. The case "Minimum Value" means that the minimum value of corresponding elements of matrices A and B should be written in corresponding element of matrix C. (For example, if A[2, 1] is -2 and B[2, 1] is -10 then C[2, 1] should be assigned -10 or if A[0, 2] is 2 and B[0, 2] is 5 then C[0, 2] should be assigned 2, etc.).

Hint: You may use the following function prototype for ***Matrix_Operations***.

*void Matrix_Operations(int *A, int *B, int *C, int OP);*

For testing, you may initialize matrices A and B (with N = 8) with some numbers. All elements of matrix C should be initialized to 0.

b) Measure the performance (execution time or number of clock cycles) for this function for N = 8. (Configure the Nios II processor *without the instruction or data caches* for this part).

c) Perform design space exploration to determine the optimum data cache and instruction cache sizes for this application (by configuring the Nios II processor with a few different cache sizes up to 16K for instruction and data caches). Report the result for at least three different configurations (including the optimum configuration) in a table. What is the optimum size for instruction and data caches for this application with N = 8?